🚀 JIRA Data Engineering Pipeline – Medallion Architecture

This project implements a high-performance Data Engineering pipeline to process JIRA data, utilizing the Medallion Architecture to transform raw data into strategic business intelligence regarding SLA (Service Level Agreement).

---

us English Version

📋 Objective

Automates JIRA ticket ingestion and processing to calculate resolution time in **business hours**, precisely excluding weekends and national holidays through **BrasilAPI** integration.

🏗️ Pipeline Architecture

The project follows the **Medallion Architecture**, ensuring data lineage and governance:

- **Bronze Layer**: Raw ingestion of JSON files, preserving source data immutability.
- **Silver Layer**: Data cleaning, schema normalization, and conversion to **Parquet** format for optimized I/O performance.
- **Gold Layer**: Business rules application and vectorized SLA calculation based on priority.
- **Data Quality (Step 4)**: Automated auditing system for integrity, volume consistency, and null validation via `data_dictionary.json`.

🧠 Core Engine Logic

The analytical heart of the solution resides in `sla_calculation.py`, designed for precision and scalability:

- `get_expected_sla(priority)`: Maps ticket priority to business requirements (High: 24h, Medium: 72h, Low: 120h) using strict conditional mapping.
- `calculate_business_hours(start, end, holidays)`: The "brain" of the engine. It uses vectorized `numpy.busday_count` to calculate the time delta, automatically subtracting weekends and national holidays retrieved from **BrasilAPI**.
- `check_compliance(actual, limit)`: Executes the final validation, returning a Boolean flag ($True$ / $False$) used for executive reporting and alerting.

🖥️ Executive Dashboard (Streamlit)

Interactive management console for performance oversight:

- **SLA Alert Engine**: Dynamic maroon alert cards and yellow warning flags for analysts performing below the **80% compliance target**.
- **Multi-Level Drill Down**: Global Ranking ➡️ Individual Analyst Profile ➡️ Issue Type Analysis ➡️ Granular Ticket Audit.
- **Transversal Visibility**: Status column (Open, Resolved, Done) integrated across all analysis levels.

🛠️ Technologies & Best Practices

- **Python 3.12** and **Pandas** for vectorized data manipulation.
- **Numpy & LRU Cache**: Optimized time-series calculations and API management.
- **Streamlit & Plotly**: Professional visualization and interactive monitoring.
- **Security**: Environment variables (`.env`) and sensitive data protection.

📊 Execution Evidence & Quality

- **Data Funnel**: 1000 raw records ➡️ 990 valid ➡️ 804 finalized tickets.
- **Performance**: Overall system compliance currently at **92.0%**.

BR Versão em Português

📋 Objetivo

O objetivo deste projeto é automatizar a ingestão e o processamento ponta a ponta de chamados do JIRA para calcular o tempo de resolução em **horas úteis**, desconsiderando finais de semana e feriados nacionais via integração com a **BrasilAPI**.

🏗️ Arquitetura do Pipeline

O projeto segue a **Arquitetura Medallion**, garantindo linhagem de dados e governança:

- **Camada Bronze**: Ingestão bruta de arquivos JSON, preservando a imutabilidade dos dados de origem.
- **Camada Silver**: Limpeza, normalização de schema e conversão para o formato **Parquet** visando performance.
- **Camada Gold**: Aplicação de regras de negócio e cálculo de SLA vetorizado.
- **Qualidade (Step 4)**: Sistema de auditoria automática de integridade, volumetria e validação de nulos via `data_dictionary.json`.

🧠 Lógica do Motor de SLA

O núcleo analítico da solução reside em `sla_calculation.py`, projetado para precisão e escalabilidade:

- `get_expected_sla(priority)`: Traduz a prioridade do chamado em requisitos de negócio (24h, 72h, 120h).
- `calculate_business_hours(start, end, holidays)`: O "cérebro" do projeto. Utiliza `numpy.busday_count` para calcular o delta de tempo, subtraindo finais de semana e feriados nacionais obtidos da **BrasilAPI**.
- `check_compliance(actual, limit)`: Executa a validação final, retornando o indicador booleano ($True$ / $False$) que alimenta os relatórios e alertas.

🖥️ Dashboard e Visualização (Streamlit)

Console interativo de gestão para tomada de decisão estratégica:

- **Motor de Alerta de SLA**: Alertas visuais dinâmicos (cards bordô e bandeiras amarelas) para analistas abaixo da **meta de 80% de conformidade**.
- **Drill-Down Multinível**: Navegação entre Rankings Globais ➡ Perfis Individuais de Analistas ➡ Tendências por Tipo de Chamado ➡ Auditoria granular de tickets.
- **Visibilidade Transversal**: Coluna de Status integrada em todos os níveis de auditoria e detalhamento.

🛠 Tecnologias e Boas Práticas

- **Python 3.12** e **Pandas**: Manipulação de dados vetorizada.
- **Numpy e LRU Cache**: Otimização de cálculos temporais e gestão de chamadas de API.
- **Streamlit & Plotly**: Visualização profissional e dashboards interativos.
- **Segurança**: Uso de variáveis de ambiente (`.env`) e proteção de dados sensíveis.

📊 Evidências de Execução e Qualidade

- **Funil de Dados**: Ingestão de 1000 registros ➡ 990 válidos ➡ 804 chamados finalizados.
- **Conformidade**: A taxa de conformidade atual do sistema é de **92.0%**.

🛠 Guia de Execução / Execution Guide

1. **Configuração**: Execute `python init_project.py` para preparar o ambiente virtual e credenciais.
2. **Processamento**:
   - Ativar venv: `venv\Scripts\activate`
   - Rodar Pipeline: `python main.py`
3. **Visualização**: Inicie o dashboard com `streamlit run app.py`.

🗂 Estrutura de Pastas / Project Structure

```
PROJECT-ROOT/
├── .pycache/               # Python cache files (Ignored)
├── data/                   # Local Storage (Bronze/Silver/Gold layers)
├── src/                    # Source Code
│   ├── bronze/             # Ingestion Layer
│   ├── silver/             # Transformation Layer
│   ├── gold/               # Business Rules Layer
│   ├── sla_calculation.py  # SLA Calculation Engine
│   └── validate_pipeline.py # Data Quality & Auditing
├── venv/                   # Python Virtual Environment
├── .env                    # Environment Variables & Credentials
├── .gitignore              # Git ignored files
├── app.py                  # Streamlit Dashboard Interface
├── init_project.py         # Project Initialization Script
├── main.py                 # Central Orchestrator
├── README.md               # Main Documentation
└── requirements.txt        # Project Dependencies
```